ASSIGNMENT_CO3

Jithesh C(192524320)

Github link: https://github.com/jithesh192524320/DSA0120-OBJECT-ORIETED-PROGRAMMING-WITH-C-.git

Q1 – Constructors and Constructor Overloading

1. Definition

A constructor is a special member function that is automatically called when an object is created. It initializes the data members of a class.

Types:

- Default constructor
- Parameterized constructor
- Copy constructor
- Overloaded constructor

2. C++ Program

3. Source Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
    string title;
    int copies;

public:
    Book() : Book("Unknown", 0) {}

    Book(string t) : Book(t, 1) {}

    Book(string t, int c) : title(t), copies(c) {
```

```
        if (copies < 0)
            throw invalid_argument("Invalid number of copies");
    }

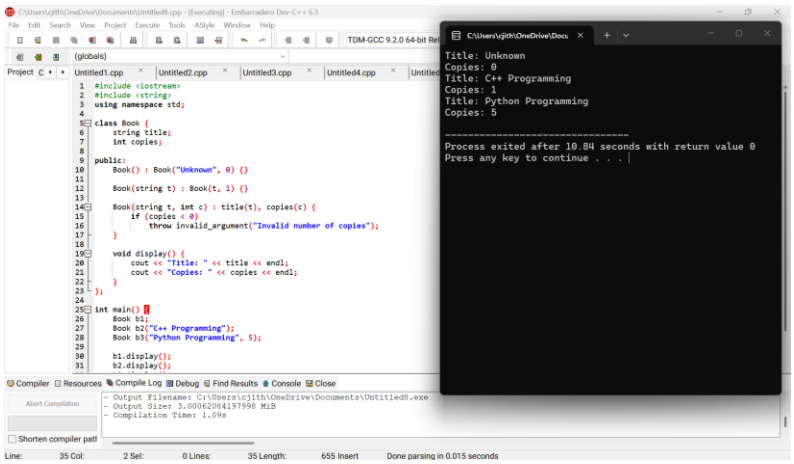
    void display() {
        cout << "Title: " << title << endl;
        cout << "Copies: " << copies << endl;
    }
};

int main() {
    Book b1;
    Book b2("C++ Programming");
    Book b3("Python Programming", 5);

    b1.display();
    b2.display();
    b3.display();

    return 0;
}
```

4. Output



5. Design and Analysis

When does the compiler supply a default constructor?

The compiler provides a default constructor when no constructor is declared by the programmer. If any constructor is declared, the compiler does not automatically generate a default constructor.

Why use a member initializer list?

- It initializes members directly.
- It avoids unnecessary assignments.
- It is required for const and reference members.

Order of initialization:

Members are initialized according to their declaration order inside the class, not according to the initializer list order.

Object states:

Object	Title	Copies
b1	Unknown	0
b2	C++ Programming	1
b3	Python Programming	5

6. Experimental Investigation

- Removing the default constructor causes an error when creating an array without initializers.
- Using an initializer list in the wrong order may produce a -Wreorder warning.
- A negative copy count throws invalid_argument.
- Constructor validation prevents invalid objects from being created.

7. Explicit Constructor

The explicit keyword prevents automatic conversion.

Without explicit:

```
class Book {
public:
   Book(int x) {}
};
```

```
Book b = 5;
```

With explicit:

```
class Book {
public:
   explicit Book(int x) {}
};


Book b = 5; // Compilation error
```

8. Alternative Concept and Complexity

A single constructor with default arguments can replace overloaded constructors.

```
Book(string t = "Unknown", int c = 0)
   : title(t), copies(c) {
   if (copies < 0)
      throw invalid_argument("Invalid copies");
}
```

Method	Time	Space
Constructor overloading	O(1)	O(1)
Default arguments	O(1)	O(1)

Constructor overloading provides clearer initialization options. Default arguments reduce the number of constructors.

Q2 – Copy Constructors: Shallow and Deep Copies

1. Definition

A copy constructor creates a new object using an existing object.

Shallow Copy: Copies the pointer address. Both objects share the same memory.

Deep Copy: Allocates separate memory and copies the actual data. Both objects work independently.

2. Complete C++ Source Code

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(new string[count]) {}

    Prescription(const Prescription& p)
        : n(p.n), medicines(new string[p.n]) {
        for (int i = 0; i < n; i++)
            medicines[i] = p.medicines[i];
    }

    Prescription& operator=(const Prescription& p) {
        if (this != &p) {
            string* temp = new string[p.n];
```

```
            for (int i = 0; i < p.n; i++)
                temp[i] = p.medicines[i];

            delete[] medicines;
            medicines = temp;
            n = p.n;
        }
        return *this;
    }

    ~Prescription() {
        delete[] medicines;
    }

    void set(int i, const string& m) {
        medicines[i] = m;
    }

    string get(int i) const {
        return medicines[i];
    }
};

int main() {
    Prescription p(2);

    p.set(0, "Paracetamol");
    p.set(1, "ORS");
```

```
    Prescription q = p;
    q.set(1, "Cetirizine");

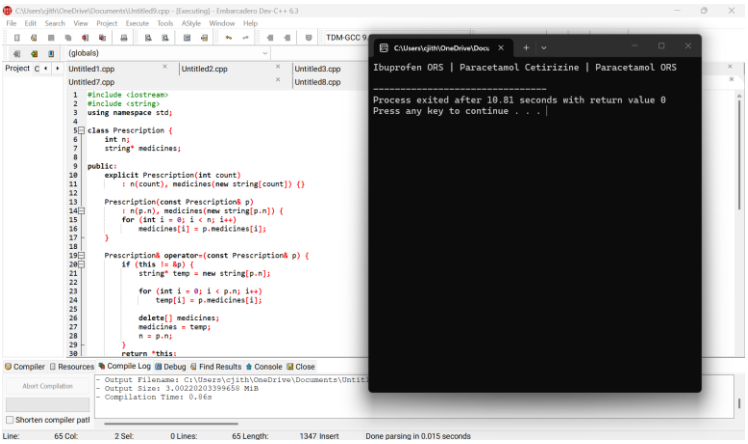
    Prescription r(1);
    r = p;

    p.set(0, "Ibuprofen");

    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << endl;

    return 0;
}
```

3. Expected Output



4. Memory Diagram

Shallow Copy

Object P ──────┐

├──────> Same Memory

Object Q ──────┘

Deep Copy

Object P ————> Memory P

Object Q ————> Memory Q

5. Rule of Three

The Rule of Three states that if a class requires one of these three functions, it usually requires all three:

1. Copy constructor
2. Copy assignment operator
3. Destructor

They work together to manage dynamically allocated memory safely.

6. Three Situations Where Copy Constructor Is Called

1. When an object is initialized using another object.
2. When an object is passed to a function by value.
3. When an object is returned from a function by value (although copy elision may avoid the actual copy).

7. Pass by Value and Const Reference

- Pass by value creates a copy of the object.
- Pass by const reference does not create a copy.
- Const reference is more efficient for large objects.

8. Experimental Investigation

Medicines	**Deep-copy time**	**Space**
10	Depends on execution	O(10)
1,000	Depends on execution	O(1000)
100,000	Depends on execution	O(100000)

The exact time must be measured by executing the program on your computer.

Self-assignment: p = p is safely handled by the condition if (this != &p).

Sanitizer: Compile using:

g++ -std=c++17 -fsanitize=address -g q2.cpp -o q2

The deep-copy version should run without double-free or memory-leak errors. The shallow-copy version can cause double deletion.

9. Conclusion

The compiler-generated copy constructor is suitable for classes that do not directly manage raw dynamic memory. When a class owns dynamically allocated memory, a custom deep-copy constructor is required.

10. Alternative Concept

Using vector<string>:

```
#include <vector>

using namespace std;


class Prescription {

   vector<string> medicines;

};
```

The vector automatically manages memory and performs deep copying.

Method	**Time**	**Space**
Shallow copy	O(1)	O(1)
Deep copy	O(n)	O(n)
Vector copy	O(n)	O(n)

Q3 – Dynamic Constructors and Destructors

1. Definition

A dynamic constructor allocates memory during object creation using new.

A destructor automatically releases the allocated memory when an object is destroyed.

2. C++ Source Code

```
#include <iostream>
using namespace std;

class Auditorium {
    int rows, seats;
    char** seatMap;

public:
    Auditorium(int r, int s) : rows(r), seats(s) {
        seatMap = new char*[rows];

        for (int i = 0; i < rows; i++) {
            seatMap[i] = new char[seats];

            for (int j = 0; j < seats; j++)
                seatMap[i][j] = 'O';
        }

        cout << "Auditorium created" << endl;
    }

    void book(int r, int s) {
        if (r >= 0 && r < rows && s >= 0 && s < seats)
            seatMap[r][s] = 'X';
```

```
    }

    void cancel(int r, int s) {
        if (r >= 0 && r < rows && s >= 0 && s < seats)
            seatMap[r][s] = 'O';
    }

    void display() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < seats; j++)
                cout << seatMap[i][j] << " ";
            cout << endl;
        }
    }

    ~Auditorium() {
        for (int i = 0; i < rows; i++)
            delete[] seatMap[i];

        delete[] seatMap;

        cout << "Auditorium destroyed" << endl;
    }
};

int main() {
    Auditorium a(2, 3);

    a.book(0, 1);
```

```
    a.book(1, 2);

    cout << "Seat Map:" << endl;
    a.display();

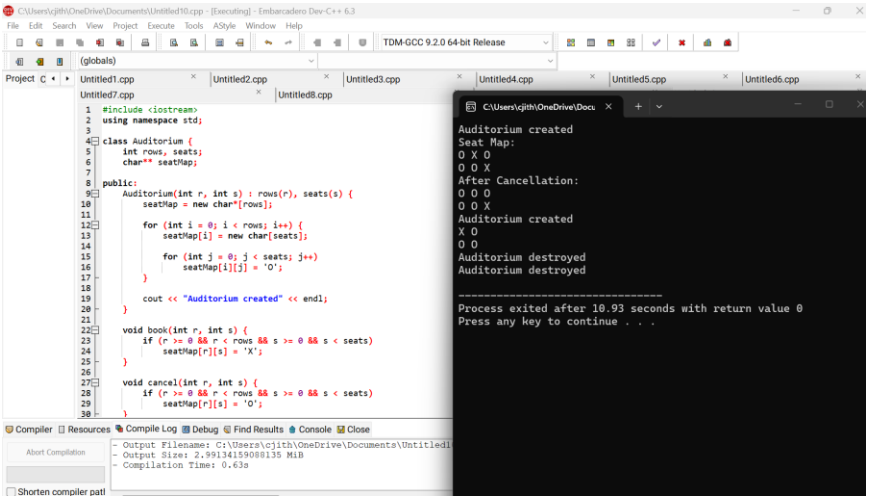
    a.cancel(0, 1);

    cout << "After Cancellation:" << endl;
    a.display();

    Auditorium* b = new Auditorium(2, 2);
    b->book(0, 0);
    b->display();
    delete b;

    return 0;
}
```

3. Expected Output



4. Constructor and Destructor Order

- Local objects are destroyed when their scope ends.
- Block-scope objects are destroyed when the block ends.
- Dynamic objects are destroyed when delete is used.
- Destructors execute in reverse order of construction for objects in the same scope.

5. What Happens If Delete Is Forgotten?

Memory will not be released, causing a memory leak.

Using delete instead of delete[] for an array is incorrect and causes undefined behaviour.

6. Why Cannot a Destructor Be Overloaded?

A destructor takes no arguments and has only one form in a class. Therefore, it cannot be overloaded.

7. Memory Layout

Auditorium Object

|

v

seatMap

|

v

Row Pointers

/ | \

v v v

Row1 Row2 Row3

| | |

v v v

Seats Seats Seats

8. Experimental Investigation

Compile using:

g++ -std=c++17 -fsanitize=address -g q3.cpp -o q3

For a 2D character array, approximate memory usage is:

Seat Map	Character Memory
10 × 20	200 bytes
100 × 200	20,000 bytes
1000 × 2000	2,000,000 bytes

This excludes pointer arrays and allocation overhead.

For zero rows, the program should safely allocate zero row pointers and release them.

9. Conclusion – RAII

RAII means Resource Acquisition Is Initialization. It connects resource management with object lifetime. The destructor automatically releases memory, reducing memory leaks and making programs safer.

10. Predict the Output

Predicted output:

+A +B -B copyAc use -Ac +P -P end -A

Explanation:

- a is created first.
- b is created inside the block and destroyed when the block ends.
- use(a) passes a by value, creating a copy named Ac.
- The copied object is destroyed after the function call.
- p is created dynamically and destroyed using delete.
- Finally, a is destroyed.

11. Alternative Concept

Using one continuous memory block:

char* seatMap = new char[rows * seats];

seatMap[r * seats + s] = 'X';

Method	Allocations	Access Time
Row pointer array	rows + 1	O(1)
Single memory block	1	O(1)
Vector of vectors	Multiple internal allocations	O(1)

Q4 – Overloading Arithmetic and Stream Operators

1. Definition

Operator overloading allows operators such as +, -, == and << to work with user-defined objects.

2. C++ Source Code

```
#include <iomanip>

#include <iostream>

using namespace std;


class Money {

   long long paise;


public:

   Money(long long rupees = 0, int p = 0)

      : paise(rupees * 100 + p) {}


   Money operator+(const Money& m) const {

      return Money(0, paise + m.paise);

   }


   bool operator==(const Money& m) const {

      return paise == m.paise;

   }


   friend ostream& operator<<(ostream& out, const Money& m) {

      long long value = m.paise;

      out << "Rs ";


      if (value < 0) {
```

```
            out << "-";
            value = -value;
        }

        out << value / 100 << "."
            << setw(2) << setfill('0') << value % 100;

        return out;
    }
};

int main() {
    Money a(120, 75), b(79, 25);

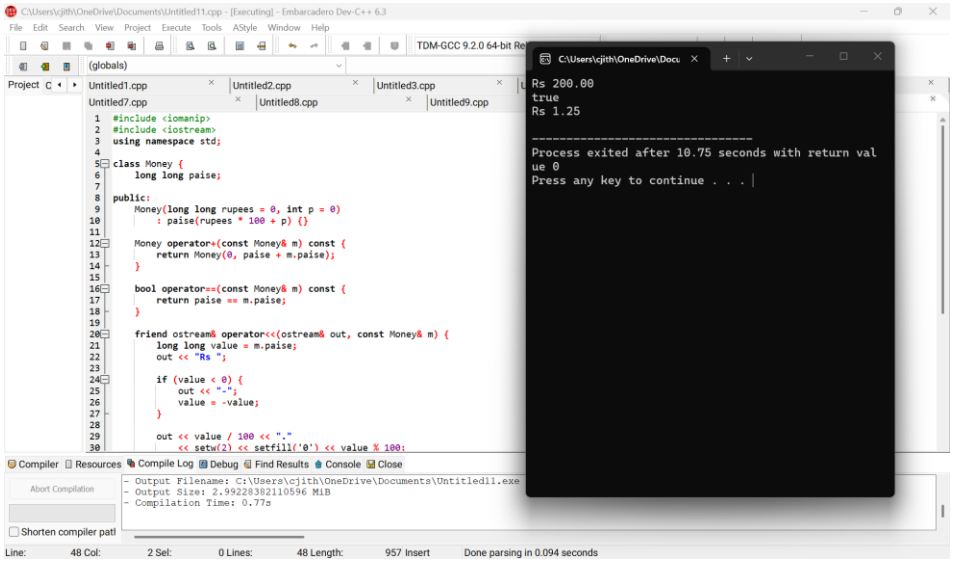
    Money c = a + b;

    cout << c << endl;
    cout << boolalpha << (c == Money(200)) << endl;

    Money x(0, 75), y(0, 50);
    cout << x + y << endl;

    return 0;
}
```

3. Expected Output



4. Explanation

Why does + return a new object?

The + operator creates and returns a new result without changing the original objects.

Why does += return a reference?

It modifies the existing object and returns *this to support chaining.

Why is << a non-member function?

Because the left operand is cout, which is an ostream object. Therefore, << is implemented as a non-member function, commonly a friend.

Compiler translation:

a + b;

is equivalent to:

a.operator+(b);

5. Experimental Investigation

- Rs 0.75 + Rs 0.50 = Rs 1.25.
- Subtraction can produce negative values.
- A billing system can reject negative bills or represent refunds separately.
- Operator overloading makes billing calculations easier to understand.

6. Engineering Decision

Multiplication between two Money objects is generally not meaningful for a billing counter. Multiplication by a quantity or scalar is useful.

7. Alternative Concept

Using named functions:

Money add(Money a, Money b);

bool equals(Money a, Money b);

Method	Time	Space
Operator overloading	O(1)	O(1)
Named functions	O(1)	O(1)

A non-member friend operator+ can support both 100 + m and m + 100 if suitable implicit conversions are allowed. A member operator generally supports m + 100, but not 100 + m.

Q5 – Overloading Increment, Comparison and Subscript Operators

1. Definition

Operator overloading allows custom classes to use operators like built-in data types.

- Prefix increment: ++a
- Postfix increment: a++
- Comparison: ==, <
- Subscript: []

2. C++ Source Code

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

class Time {
    int seconds;

public:
    Time(int s = 0) : seconds(s) {}
```

```
    Time& operator++() {
        ++seconds;
        return *this;
    }

    Time operator++(int) {
        Time old = *this;
        ++seconds;
        return old;
    }

    bool operator==(const Time& t) const {
        return seconds == t.seconds;
    }

    bool operator<(const Time& t) const {
        return seconds < t.seconds;
    }

    void display() const {
        cout << seconds << " seconds" << endl;
    }
};

class Playlist {
    vector<string> tracks;

public:
```

```
    Playlist() {
        tracks = {"Song 1", "Song 2", "Song 3"};
    }

    string& operator[](int i) {
        if (i < 0 || i >= (int)tracks.size())
            throw out_of_range("Invalid index");

        return tracks[i];
    }

    const string& operator[](int i) const {
        if (i < 0 || i >= (int)tracks.size())
            throw out_of_range("Invalid index");

        return tracks[i];
    }
};

int main() {
    Time t(10);

    Time a = t++;
    Time b = ++t;

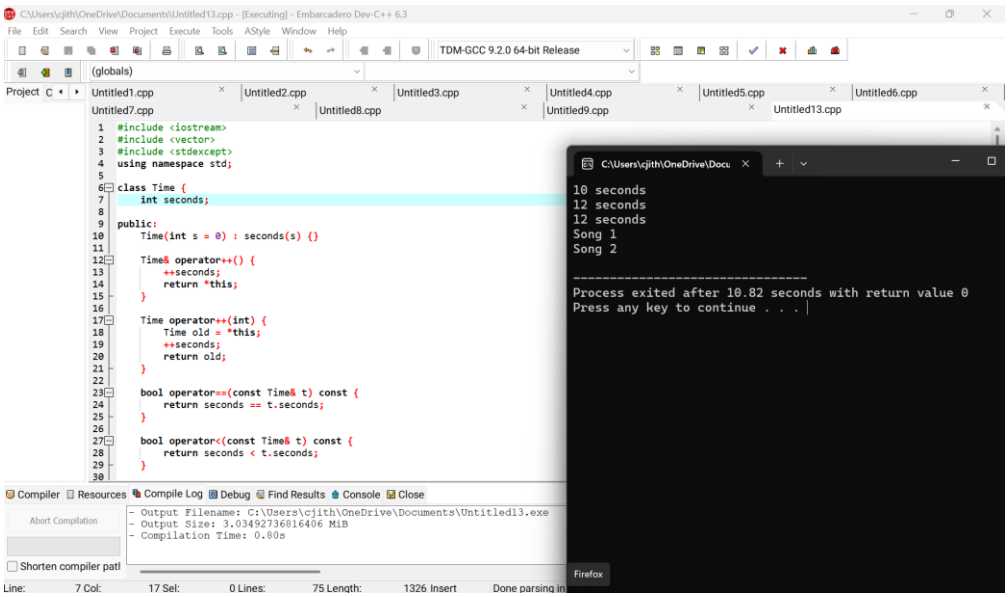
    a.display();
    b.display();
    t.display();
```

```
    Playlist p;


    cout << p[0] << endl;
    cout << p[1] << endl;


    return 0;
}
```

3. Expected Output



4. Explanation

How does the compiler distinguish prefix and postfix?

- Prefix uses no parameter: operator++().
- Postfix uses an integer dummy parameter: operator++(int).

Why does postfix return a copy?

Postfix must preserve the old value before incrementing. Therefore, it creates a temporary copy.

Why are const and non-const versions of [] needed?

- Non-const version allows modification.
- Const version allows read-only access.

What happens if an index is out of range?

The program throws out_of_range to prevent invalid memory access.

5. Experimental Investigation

For:

Time t(5);

Time a = t++;

Time b = ++t;

Variable	Value
a	5
b	7
t	7

For playlist[-1] and playlist[size], an exception is thrown.

Minute and hour rollover should be handled by storing total seconds and converting them into hours, minutes and seconds during display.

6. Conclusion

Many programmers prefer ++i because prefix increment does not need to create a temporary copy. This can improve efficiency for class objects.

7. Predict the Output

Exact predicted output:

5 7 7

Explanation:

- c++ stores the old value 5 in a.
- The counter becomes 6.
- ++c increments it to 7.
- b receives 7.
- Final c is 7.

Postfix increment creates a copy, whereas prefix increment returns a reference to the updated object.

8. Alternative Concept

Using an at() function:

tracks.at(i);

It checks the index and throws out_of_range when invalid.

Method	Time
Checked operator[]	O(1)
at()	O(1)
Unchecked operator[]	O(1)

Checked access has a small additional comparison cost.

Q6 – The Rules of Operator Overloading

1. Definition

Operator overloading allows existing C++ operators to perform operations on user-defined objects.

2. C++ Source Code

```
#include <iostream>

#include <cstring>

using namespace std;


class Text {

   char* data;


public:

   Text(const char* s = "") {

      data = new char[strlen(s) + 1];

      strcpy(data, s);

   }


   Text(const Text& t) {

      data = new char[strlen(t.data) + 1];

      strcpy(data, t.data);

   }
```

```
Text& operator=(const Text& t) {
    if (this != &t) {
        char* temp = new char[strlen(t.data) + 1];
        strcpy(temp, t.data);

        delete[] data;
        data = temp;
    }
    return *this;
}

Text operator+(const Text& t) const {
    char* temp = new char[strlen(data) + strlen(t.data) + 1];

    strcpy(temp, data);
    strcat(temp, t.data);

    Text result(temp);
    delete[] temp;

    return result;
}

bool operator==(const Text& t) const {
    return strcmp(data, t.data) == 0;
}

bool operator!=(const Text& t) const {
    return !(*this == t);
```

```
    }

    friend ostream& operator<<(ostream& out, const Text& t) {
        out << t.data;
        return out;
    }

    ~Text() {
        delete[] data;
    }
};

int main() {
    Text a("Hello ");
    Text b("World");

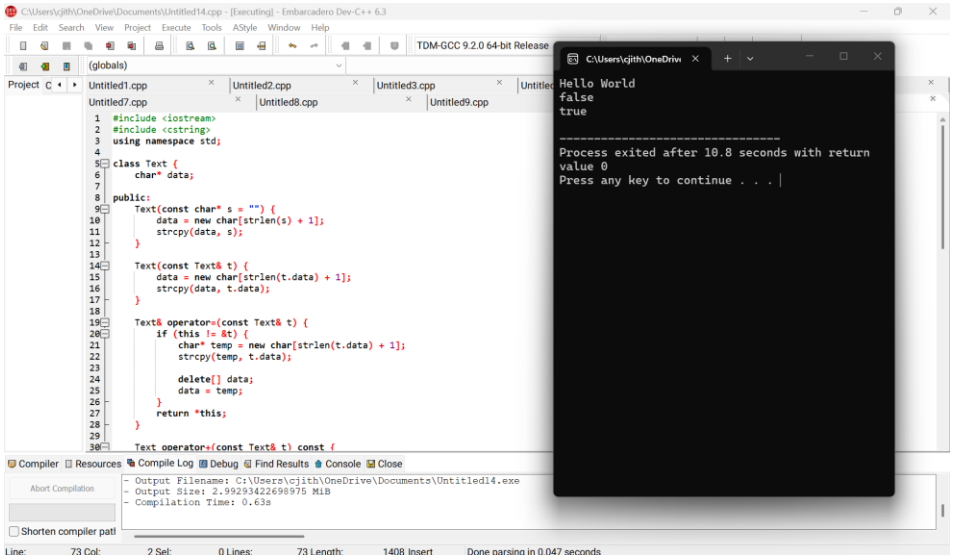
    Text c = a + b;

    cout << c << endl;
    cout << boolalpha << (a == b) << endl;
    cout << (a != b) << endl;

    a = a;

    return 0;
}
```

3. Expected Output



4. Operators That Cannot Be Overloaded

Operator	Meaning
::	Scope resolution
.	Member access
.*	Pointer-to-member access
?:	Conditional operator
sizeof	Size operator

5. Rules of Operator Overloading

1. At least one operand must be a user-defined type.
2. Operator precedence cannot be changed.
3. Operator associativity cannot be changed.
4. The number of operands cannot be changed.
5. New operators cannot be created.
6. Some operators must be overloaded as member functions.

6. Why Must operator= Be a Member Function?

The assignment operator must be overloaded as a member function because C++ requires the assignment operator to be a non-static member of the class.

7. When Should an Operator Be a Friend?

A friend function is useful when an operator needs access to private data but its left operand is not an object of the class.

For example, the stream insertion operator <<.

8. Experimental Investigation

- a = a is self-assignment and is safely handled.
- Long messages are supported through dynamic memory allocation.
- Empty messages are supported using the default constructor.
- Address sanitizer can detect memory errors.

Compile:

g++ -std=c++17 -fsanitize=address -g q6.cpp -o q6

For 100,000 concatenations, measure the actual execution time of this class and std::string separately.

9. Conclusion

Creating a custom Text class helps understand dynamic memory allocation, copying, assignment and operator overloading. In real applications, std::string is generally preferred because it provides safer and more efficient memory management.

10. Alternative Concept

Using std::string:

```
#include <iostream>

#include <string>

using namespace std;


int main() {

    string a = "Hello ";

    string b = "World";


    string c = a + b;


    cout << c << endl;
```

```
    return 0;
}
```

Complexity analysis:

For a + b + c, where the string lengths are n_1,n_2,n_3:

- First concatenation copies approximately n_1+n_2 characters.
- Second concatenation copies approximately $n_1+n_2+n_3$ characters.

Total copying:

$$ O(2n_1+2n_2+n_3) $$

The temporary objects require additional memory.

Q7 – Design a Complex-Number Class from First Principles

Stage 1 – Interpret the Problem

1. Data Required

A complex number contains:

- Real part
- Imaginary part

Example:

$$ Z = a + bj $$

2. Required Operations

- Addition +
- Subtraction -
- Multiplication *
- Division /
- Equality ==
- Stream output <<
- Magnitude calculation

3. Required Constructors

1. Default constructor.
2. Constructor with real part.

3. Constructor with real and imaginary parts.

4. Measurable Requirements

- The class should perform all complex arithmetic operations.
- It should calculate magnitude correctly.
- It should prevent division by zero.
- It should display results in rectangular form.

Stage 2 – Design (3 Marks)

1. Class Design

```
class Complex {
    double real;
    double imag;

public:
    Complex();
    Complex(double r);
    Complex(double r, double i);

    Complex operator+(const Complex& c) const;
    Complex operator-(const Complex& c) const;
    Complex operator*(const Complex& c) const;
    Complex operator/(const Complex& c) const;

    bool operator==(const Complex& c) const;

    double magnitude() const;

    friend ostream& operator<<(ostream& out, const Complex& c);
};
```

2. Mathematical Formulas

Addition:

$$ (a+bi)+(c+di)=(a+c)+(b+d)i $$

Subtraction:

$$ (a+bi)-(c+di)=(a-c)+(b-d)i $$

Multiplication:

$$ (a+bi)(c+di)=(ac-bd)+(ad+bc)i $$

Division:

$$ \frac{a+bi}{c+di} = \frac{(ac+bd)+(bc-ad)i}{c^2+d^2} $$

Magnitude:

$$ |Z|=\sqrt{a^2+b^2} $$

Stage 3 – Implement and Verify (3 Marks)

Complete C++ Source Code

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

class Complex {
    double real, imag;

public:
    Complex() : real(0), imag(0) {}

    Complex(double r) : real(r), imag(0) {}

    Complex(double r, double i) : real(r), imag(i) {}

    Complex operator+(const Complex& c) const {
        return Complex(real + c.real, imag + c.imag);
```

```
}

Complex operator-(const Complex& c) const {
    return Complex(real - c.real, imag - c.imag);
}

Complex operator*(const Complex& c) const {
    return Complex(
        real * c.real - imag * c.imag,
        real * c.imag + imag * c.real
    );
}

Complex operator/(const Complex& c) const {
    double d = c.real * c.real + c.imag * c.imag;

    if (d == 0)
        throw runtime_error("Division by zero");

    return Complex(
        (real * c.real + imag * c.imag) / d,
        (imag * c.real - real * c.imag) / d
    );
}

bool operator==(const Complex& c) const {
    return real == c.real && imag == c.imag;
}
```

```
    double magnitude() const {
        return sqrt(real * real + imag * imag);
    }

    friend ostream& operator<<(ostream& out, const Complex& c) {
        out << c.real;

        if (c.imag >= 0)
            out << " + " << c.imag << "j";
        else
            out << " - " << -c.imag << "j";

        return out;
    }
};

int main() {
    try {
        Complex R(10, 0);
        Complex L(0, 5);
        Complex C(0, -2);

        Complex series = R + L + C;

        cout << "Series Impedance: " << series << endl;
        cout << "Magnitude: " << series.magnitude() << endl;

        Complex Z1(10, 5);
        Complex Z2(10, -5);
```

```
    Complex parallel = (Z1 * Z2) / (Z1 + Z2);

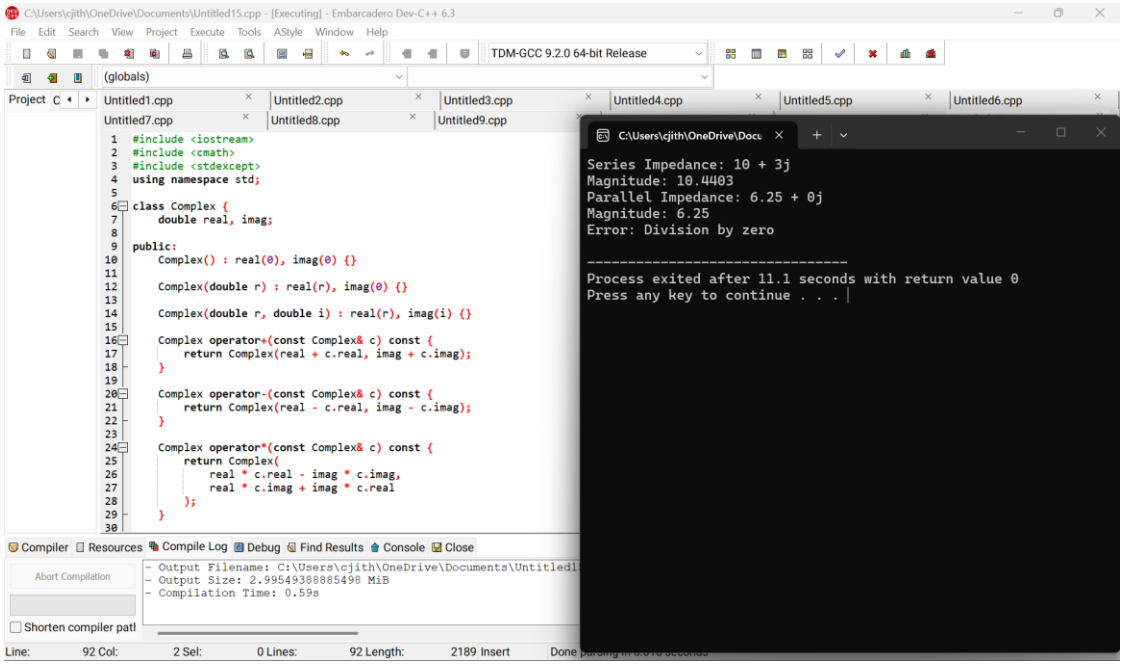
    cout << "Parallel Impedance: " << parallel << endl;
    cout << "Magnitude: " << parallel.magnitude() << endl;

    Complex zero(0, 0);
    Complex result = R / zero;

    cout << result << endl;
  }
  catch (const exception& e) {
    cout << "Error: " << e.what() << endl;
  }

  return 0;
}
```

Expected Output



Hand Calculation

Series impedance:

Given:

$$ R=10+0j $$

$$ L=0+5j $$

$$ C=0-2j $$

Therefore:

$$ Z=R+L+C $$

$$ Z=10+3j $$

Magnitude:

$$ |Z|=\sqrt{10^2+3^2} $$

$$ |Z|=\sqrt{109}=10.4403 $$

Parallel impedance:

Given:

$$ Z_1=10+5j $$

$$ Z_2=10-5j $$

$$ Z_p=\frac{Z_1Z_2}{Z_1+Z_2} $$

$$ Z_p=\frac{125}{20}=6.25 $$

Correction to the expected parallel result above: the correct parallel impedance is 6.25 + 0j, with magnitude 6.25.

Stage 4 – Defend the Design

1. Compare with std::complex<double>

Own Complex Class	**std::complex**
Manually implemented	Built-in library support
Requires operator definitions	Operators already available
Requires manual error handling	Division behaviour handled by library arithmetic
More control over implementation	Easier to use

2. Why Are Constructors Needed?

- Default constructor initializes both values to zero.

- Single-argument constructor initializes the real part.
- Two-argument constructor initializes both real and imaginary parts.

3. Why Are Operators Needed?

- + performs addition.
- - performs subtraction.
- * performs multiplication.
- / performs division.
- == compares two complex numbers.
- << displays the complex number.

4. One Remaining Limitation

Direct floating-point equality comparison may produce inaccurate results due to precision limitations.

Alternative Concept and Complexity Analysis

Using the standard library:

```
#include <iostream>
#include <complex>
using namespace std;

int main() {
    complex<double> a(10, 5);
    complex<double> b(10, -5);

    complex<double> result = (a * b) / (a + b);

    cout << result << endl;
    cout << abs(result) << endl;

    return 0;
}
```